Федеральное государственное автономное образовательное учреждение высшего образования «Национальный исследовательский университет ИТМО»

Факультет программной инженерии и компьютерной техники

Дисциплина «Тестирование программного обеспечения»

Лабораторная работа № 1

Вариант № 330917

Выполнил:

Кирячек Тимофей Алексеевич

P3309

Преподаватель:

Наумова Надежда Александровна

Санкт-Петербург, 2026

Задание

Лабораторная работа #1

1. Для указанной функции провести модульное тестирование разложения функции в степенной ряд. Выбрать достаточное тестовое покрытие.
2. Провести модульное тестирование указанного алгоритма. Для этого выбрать характерные точки внутри алгоритма, и для предложенных самостоятельно наборов исходных данных записать последовательность попадания в характерные точки. Сравнить последовательность попадания с эталонной.
3. Сформировать доменную модель для заданного текста. Разработать тестовое покрытие для данной доменной модели

Введите вариант: 330917

1. Функция arctg(x)
2. Программный модуль для работы с расширяющим деревом (http://www.cs.usfca.edu/~galles/visualization/SplayTree.html)
3. Описание предметной области:

> Марвин наблюдал за ней с холодным презрением. Его логические схемы в отвращении застрекотали и начали манипулировать идеей применить к двери физическое насилие. Затем они щелкнули и сказали: "А стоит ли обращать внимание? Ради чего?". Тогда схемы решили позабавиться, произведя сравнительный анализ молекулярных компонентов двери и клеток человеческого мозга. После этого они на бис измерили уровень водорода в близлежащем кубическом парсеке космоса, а потом им все надоело, и они отключились. С судорогой отчаяния робот повернулся.

Выполнение

Часть 1

Функция arctg(x)

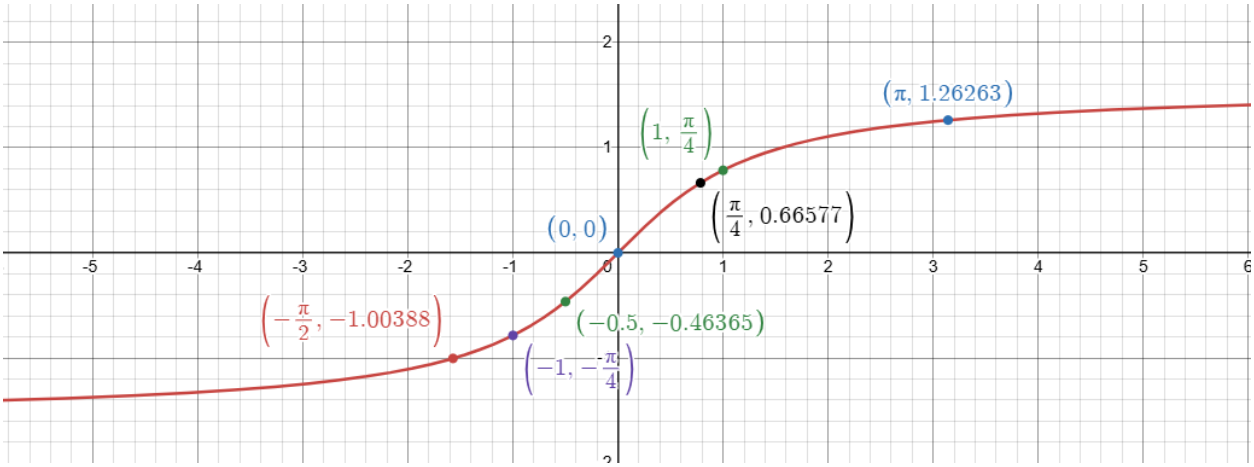


Таблица тестовых значений

№	x	arctan(x)	Пояснение
1	0	1	Начало периода, максимум функции.
2	-1000	-1.5698	Большое отрицательное число -> -π/2
3	-10	-1.4711	Близко к -π/2
4	-1	-0.7854	Равно -π/4
5	-0.999999	-0.7854	X близок к -1 -> arctan(x) около -π/4
6	-0.5	-0.4636	Произвольное отрицательное значение
7	-0.00001	-0.00001	Очень маленькое отрицательное число (проверка около 0)
8	0	0	Ноль
9	1000	1.5698	Большое отрицательное число -> π/2
10	10	1.4711	Близко к π/2
11	1	0.7854	Равно π/4
12	0.999999	0.7854	X близок к 1 -> arctan(x) около π/4
13	0.5	0.4636	Произвольное положительное значение
14	0.00001	0.00001	Очень маленькое положительное число (проверка около 0)
15	π/2 = 1.5708	1.0039	Угол 90 как аргумент
16	π/4 = 0.7854	0.6658	Угол 45 как аргумент
17	π = 3ю1316	1ю2626	Угол 180 как аргумент
18	Double.NaN	NaN	Некорректное значение
19	+∞	1.5708	Предел -> π/2
20	-∞	-1.5708	Предел -> -π/2

Результаты:

TrigonometricTest

all > lab1.task1.utils > TrigonometricTest

44	0	0	0.472s
tests	failures	ignored	duration

100% successful

Tests

Test	Method name	Duration	Result
arctg(0) = 0.0	checkBetweenDots(double, double)[10]	0.001s	passed
arctg(0.1) = 0.09966865249116204	checkBetweenDots(double, double)[11]	0.001s	passed
arctg(0.2) = 0.1973955598498808	checkBetweenDots(double, double)[12]	0.001s	passed
arctg(0.3) = 0.2914567944778671	checkBetweenDots(double, double)[13]	0.001s	passed
arctg(0.4) = 0.3805063771123649	checkBetweenDots(double, double)[14]	0.002s	passed
arctg(0.5) = 0.4636476090008061	checkBetweenDots(double, double)[15]	0.001s	passed
arctg(0.6) = 0.5404195002705842	checkBetweenDots(double, double)[16]	0.002s	passed
arctg(0.7) = 0.6107259643892086	checkBetweenDots(double, double)[17]	0.001s	passed
arctg(0.8) = 0.6747409422235527	checkBetweenDots(double, double)[18]	0.001s	passed
arctg(0.9) = 0.7328151017865066	checkBetweenDots(double, double)[19]	0.001s	passed
arctg(-0.9) = -0.7328151017865066	checkBetweenDots(double, double)[1]	0.002s	passed
arctg(1) = 0.7853981633974483	checkBetweenDots(double, double)[20]	0.001s	passed
arctg(-0.8) = -0.6747409422235527	checkBetweenDots(double, double)[2]	0.002s	passed
arctg(-0.7) = -0.6107259643892086	checkBetweenDots(double, double)[3]	0.002s	passed
arctg(-0.6) = -0.5404195002705842	checkBetweenDots(double, double)[4]	0.002s	passed
arctg(-0.5) = -0.4636476090008061	checkBetweenDots(double, double)[5]	0.001s	passed
arctg(-0.4) = -0.3805063771123649	checkBetweenDots(double, double)[6]	0.001s	passed
arctg(-0.3) = -0.2914567944778671	checkBetweenDots(double, double)[7]	0.001s	passed
arctg(-0.2) = -0.1973955598498808	checkBetweenDots(double, double)[8]	0.001s	passed
arctg(-0.1) = -0.09966865249116204	checkBetweenDots(double, double)[9]	0.001s	passed

Покрытие:

task1 > lab1.task1.utils

lab1.task1.utils

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
Trigonometric.java		97%		93%	2	11	1	20	1	3	0	1
Total	3 of 106	97%	1 of 16	93%	2	11	1	20	1	3	0	1

Часть 2

Расширяющее дерево (Splay Tree) — это самобалансирующееся бинарное дерево поиска, в котором после каждой операции доступа (вставка, поиск, удаление) выполняется операция **splay** — перемещение использованного узла в корень дерева с помощью вращений. В отличие от красно-чёрного дерева, Splay Tree не хранит дополнительную информацию о цветах или высоте, а балансировка достигается динамически за счёт перемещений узлов.

Оно гарантирует амортизированную сложность O(log n) для операций:

- Вставка элемента
- Удаление элемента
- Поиск элемента

Цель тестирования — убедиться, что:

- дерево корректно выполняет операции вставки, удаления и поиска случайных элементов,
- операция splay корректно поднимает узел к корню,
- сохраняются свойства бинарного дерева поиска,
- структура дерева остаётся корректной после любой последовательности операций,
- вспомогательные методы (count, print, validation) работают правильно.

Результаты:

task2 > lab1.task2.datastructures

lab1.task2.datastructures

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
SplayTree		99%		95%	5	67	0	136	0	17	0	1
SplayTree.Node		100%		n/a	0	1	0	4	0	1	0	1
Total	2 of 526	99%	5 of 100	95%	5	68	0	140	0	18	0	2

task2 > lab1.task2.datastructures > SplayTree

SplayTree

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods
leftRotate(SplayTree.Node)		97%		87%	1	5	0	13	0	1
rightRotate(SplayTree.Node)		97%		87%	1	5	0	13	0	1
splay(SplayTree.Node)		100%		94%	1	10	0	21	0	1
remove(int)		100%		95%	1	12	0	21	0	1
insert(int)		100%		100%	0	5	0	19	0	1
printTree(SplayTree.Node, String, boolean)		100%		100%	0	3	0	11	0	1
isValidBST(SplayTree.Node, int, int)		100%		100%	0	6	0	5	0	1
search(int)		100%		100%	0	4	0	9	0	1
checkParents(SplayTree.Node, SplayTree.Node)		100%		87%	1	5	0	5	0	1
inOrder(SplayTree.Node, List)		100%		100%	0	2	0	6	0	1
isValidSplayTree()		100%		100%	0	3	0	2	0	1
countNodes(SplayTree.Node)		100%		100%	0	2	0	3	0	1
inOrder()		100%		n/a	0	1	0	3	0	1
printTree()		100%		n/a	0	1	0	2	0	1
countNodes()		100%		n/a	0	1	0	1	0	1
SplayTree()		100%		n/a	0	1	0	1	0	1
getRoot()		100%		n/a	0	1	0	1	0	1
Total	2 of 517	99%	5 of 100	95%	5	67	0	136	0	17

SplayTreeTest

all > lab1.task2.datastructures > SplayTreeTest

20	0	0	0.125s
tests	failures	ignored	duration

100% successful

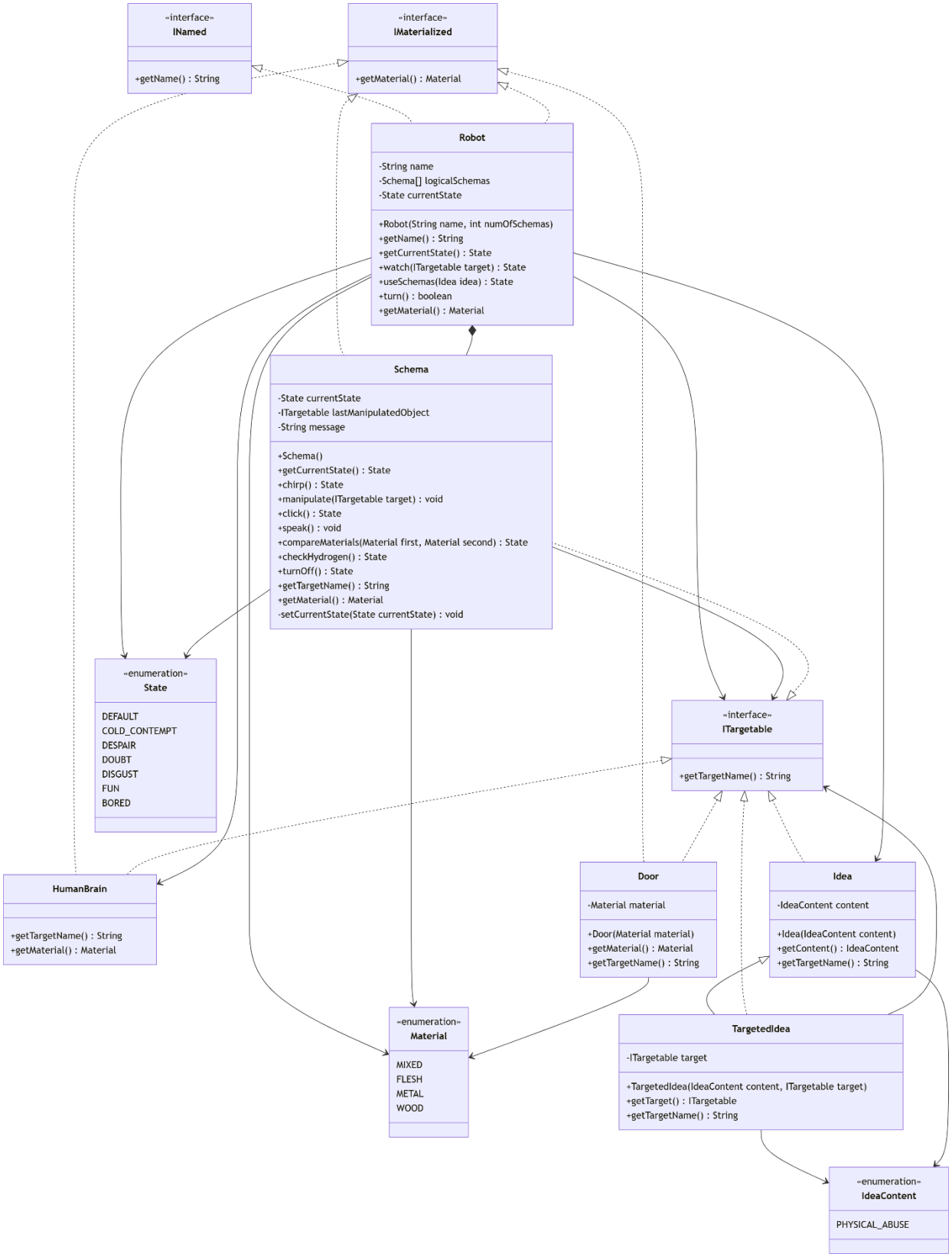
Tests

Test	Method name	Duration	Result
Count nodes	testCountNodes()	0.004s	passed
Insert + inorder correctness	testInsertAndInOrder()	0.003s	passed
Insert: new node becomes root (splay)	testInsertMakesNodeRoot()	0.003s	passed
Validation fails: BST property violated	testInvalidBST()	0.001s	passed
Validation fails: wrong parent pointer	testInvalidParentPointer()	0.003s	passed
Tree validation check	testIsValidSplayTree()	0.001s	passed
Many operations in a row	testManyOperations()	0.003s	passed
printTree works	testPrintTree()	0.001s	passed
Remove from empty tree does nothing and NOT throws anything	testRemoveFromEmptyTree()	0.001s	passed
Remove leaf	testRemoveLeaf()	0.001s	passed
Remove not found	testRemoveNotFound()	0.001s	passed
Remove node with one child	testRemoveOneChild()	0.001s	passed
Remove root	testRemoveRoot()	0.002s	passed
Remove node with two children	testRemoveTwoChildren()	0.002s	passed
Remove node with left child	testRemoveWithLeftChild()	0.009s	passed
Search not found	testSearchNotFound()	0.083s	passed
Search splays node to root	testSearchSplaysToRoot()	0.002s	passed
Zig case	testZigCase()	0.001s	passed
Zig-Zag case	testZigZagCase()	0.001s	passed
Zig-Zig case	testZigZigCase()	0.002s	passed

Часть 3

Марвин наблюдал за ней с холодным презрением. Его логические схемы в отвращении застрекотали и начали манипулировать идеей применить к двери физическое насилие. Затем они щелкнули и сказали: "А стоит ли обращать внимание? Ради чего?". Тогда схемы решили позабавиться, произведя сравнительный анализ молекулярных компонентов двери и клеток человеческого мозга. После этого они на бис измерили уровень водорода в близлежащем

кубическом парсеке космоса, а потом им все надоело, и они отключились. С судорогой отчаяния робот повернулся .



Описание классов:

Robot - механическое существо с именем и набором логических схем. Может находиться в различных эмоциональных состояниях. Наблюдает за объектами, обрабатывает идеи через свои схемы. Способен разворачиваться. Состоит из смешанных материалов.

Schema - мыслительный модуль робота с изменяемым состоянием. Может чирикать, щёлкать, говорить, манипулировать объектами, сравнивать материалы и проверять водород. Последовательно меняет состояния при операциях. Состоит из смешанных материалов.

HumanBrain - органический орган, используемый как эталон для сравнения материалов. Состоит из плоти.

Door - физический объект из определённого материала.

Idea - абстрактная концепция с определённым содержанием.

TargetedIdea - идея, имеющая конкретный целевой объект.

State - перечисление эмоциональных состояний (DEFAULT, COLD_CONTEMPT, DESPAIR, DOUBT, DISGUST, FUN, BORED)

Material - перечисление типов материалов (MIXED, FLESH)

IdeaContent - перечисление содержания идей (PHYSICAL_ABUSE)

task3 > lab1.task3.implementations

lab1.task3.implementations

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
Schema		96%		83%	3	24	2	41	0	12	0	1
HumanBrain		55%		n/a	1	3	1	3	1	3	0	1
Robot		100%		96%	1	22	0	45	0	7	0	1
Door		100%		n/a	0	3	0	5	0	3	0	1
Total	12 of 533	97%	4 of 48	91%	5	52	3	94	1	25	0	4

Package lab1.task3.implementations

all > lab1.task3.implementations

20	0	0	0.187s
tests	failures	ignored	duration

100%
successful

Classes

Class	Tests	Failures	Ignored	Duration	Success rate
RobotTest	10	0	0	0.172s	100%
SchemaTest	10	0	0	0.015s	100%

Вывод

В ходе данной лабораторной работы, я научился проводить модульное тестирование простой математической функции, алгоритма и доменной модели, имитирующую часть работы вымышленного мира вселенной “Автостопом по галактикам”.